Lecture 22: Autoencoders

A network that learns to copy - and why the copy is the point

From 784 numbers to 2, and back

One MNIST digit is a vector of **784** numbers (28×28 pixels). Could you describe it with just **2**? Think of describing a *person* by their age and height - lossy, but wonderfully compact.

Predict: can a network learn that squeeze *and* rebuild the digit from the 2 numbers - with *no labels*, ever?

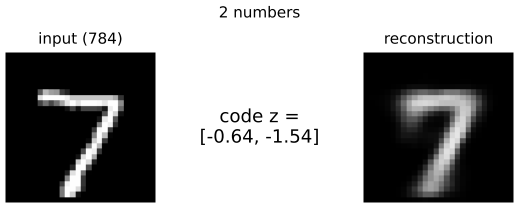
From 784 numbers to 2, and back

One MNIST digit is a vector of **784** numbers (28×28 pixels). Could you describe it with just **2**? Think of describing a *person* by their age and height - lossy, but wonderfully compact.

Predict: can a network learn that squeeze *and* rebuild the digit from the 2 numbers - with *no labels*, ever?

Basically yes (right). The trick: the network's **target is its own input**. No human labels - it **supervises itself**.

Chapters 5-7 (nets, CNNs, RNNs) were *supervised*. Autoencoders take us back to **unsupervised** learning - the territory of clustering (ch4) and PCA (the dimensionality-reduction chapter) - now with a neural net and no y.



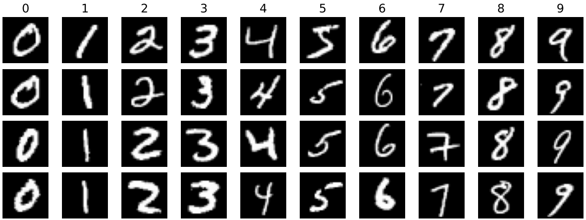
Meet the data: MNIST

Our running dataset: **MNIST** - 70,000 handwritten digits, each 28×28 grayscale ($= 784$ numbers).

- **LeCun, Cortes & Burges** (1998), from US Census / high-school forms.
- The *"hello world" of computer vision* - the first benchmark for any new method.
- Small, clean, CPU-friendly.

An autoencoder only looks at the **pixels**; the labels (0-9) are just for the pictures.

And not just images: an autoencoder works on *any* vector - tabular rows, sensor time-series, word or audio embeddings.



Outline

The autoencoder idea

Autoencoders = nonlinear PCA

Regularized autoencoders

Anomaly detection

Sparse autoencoders and interpretability

Why care? Autoencoders are everywhere

Before we build them: the ideas in this short chapter are load-bearing in today's biggest models.

- **Self-supervised pretraining. BERT** masks words and rebuilds them; **MAE** masks image patches - "masked modeling" is an autoencoder idea (denoising), and the recipe behind most foundation models.
- **LLM interpretability.** A wide **sparse autoencoder** on a model's activations splits them into single-concept features - how Anthropic reads what is inside Claude (*Scaling Monosemanticity*, 2024).
- **Cheaper inference. DeepSeek**'s Multi-head Latent Attention squeezes the KV cache into a small **latent vector** ($\sim$4% of the size), so long contexts fit in memory (DeepSeek-V3, 2024).
- **Image generation.** The **VAE** (next lecture) is the compression layer inside Stable Diffusion and Midjourney.

Encoder, bottleneck, denoising, sparse - we build each piece below; watch for where it powers the models above.

The autoencoder idea

Learn to rebuild your input, and let the bottleneck do the teaching.

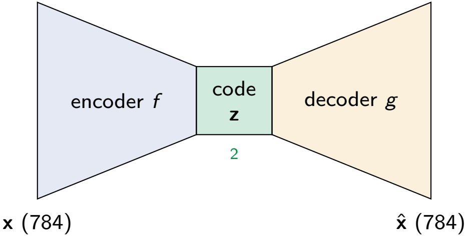
Encoder, code, decoder

An **autoencoder** is two networks trained together:

- **Encoder** $f : \mathbf{x} \mapsto \mathbf{z}$ - compress the input into a small **code z**.
- **Decoder** $g : \mathbf{z} \mapsto \hat{\mathbf{x}}$ - rebuild the input from that code.

Train so $\hat{\mathbf{x}} \approx \mathbf{x}$; the target is the input itself, so it needs **no labels** (self-supervised).

Code / latent. The code **z** is a compact summary of the input; the space it lives in is the **latent space** ("latent" = hidden / underlying). We call **z** a *code* because the encoder **encodes x** into it - a short "password" that stands for the whole image.

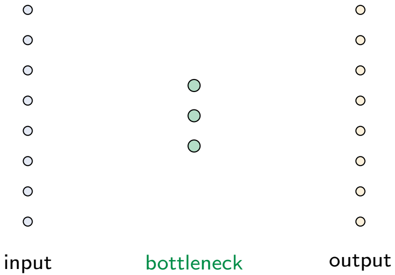


The bottleneck is the whole point

Make the code **smaller** than the input: $\dim(\mathbf{z}) < \dim(\mathbf{x})$ (**undercomplete**). The network cannot pass everything through, so it must keep only what matters - the structure the digits share.

No bottleneck, no learning. If $\dim(\mathbf{z}) \geq \dim(\mathbf{x})$ the network can just learn the **identity** - a perfect, useless copy that memorized nothing.

The **loss** is nothing new: squared error $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$ (or cross-entropy for $[0, 1]$ pixels). *Callback:* the L2 loss from regression.

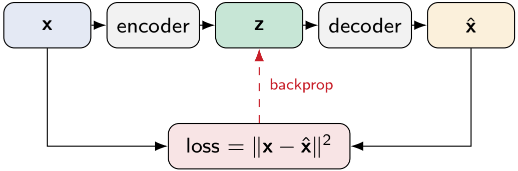


How an autoencoder learns

It is the same training loop as any net (ch5, L15) - only the *target* is unusual:

1. **Forward:** encode $\mathbf{z} = f(\mathbf{x})$, then decode $\hat{\mathbf{x}} = g(\mathbf{z})$.
2. **Loss:** compare the output to the **input itself**, $L = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$.
3. **Backward:** backprop L through decoder *and* encoder.
4. **Update:** one Adam step; repeat over mini-batches.

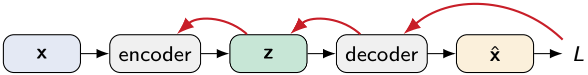
The one twist vs a normal net: the label *is* the input, so there is nothing to hand-label. Everything else - forward, backprop, Adam - is exactly ch5.



Backprop, in symbols

$\hat{\mathbf{x}} = g\big(f(\mathbf{x})\big)$ is a **composition**, so the chain rule sends the error $L = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$ back through **both** halves (θ_f, θ_g = encoder / decoder weights):

$$\underbrace{\frac{\partial L}{\partial \theta_g}}_{\text{decoder}} = \frac{\partial L}{\partial \hat{\mathbf{x}}} \frac{\partial \hat{\mathbf{x}}}{\partial \theta_g}, \qquad \underbrace{\frac{\partial L}{\partial \theta_f}}_{\text{encoder}} = \underbrace{\frac{\partial L}{\partial \hat{\mathbf{x}}} \frac{\partial \hat{\mathbf{x}}}{\partial \mathbf{z}}}_{\text{back through the decoder}} \frac{\partial \mathbf{z}}{\partial \theta_f}.$$



gradient flows back: through the decoder → the code → the encoder

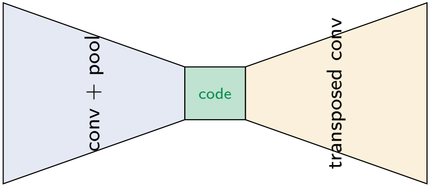
The **encoder** is trained *through* the **decoder**: the decoder first says "this code would rebuild **x** better", then the encoder learns to produce such codes - which is why the two halves share one representation. `loss.backward()` does it all.

You already built the decoder

For images, the two halves are just a **CNN and its mirror** - a **convolutional autoencoder**:

- Encoder = convolution + pooling (*ch6, L16*) - shrink to a code.
- Decoder = **transposed convolution** (*L19*) - grow the code back to an image.

Nothing new to learn here: you built both halves in the CNN chapter. An autoencoder just points them at each other.



In practice (PyTorch)

A working autoencoder is a handful of lines - encoder, decoder, and the copy-yourself loop:

```
enc = nn.Sequential(
  nn.Linear(784,256), nn.ReLU(),
  nn.Linear(256,32))          # code
dec = nn.Sequential(
  nn.Linear(32,256), nn.ReLU(),
  nn.Linear(256,784), nn.Sigmoid())

for xb in loader:              # no labels!
  xh = dec(enc(xb))
  loss = F.mse_loss(xh, xb)
  opt.zero_grad()
  loss.backward(); opt.step()
```

Practical notes

- Scale pixels to $[0, 1]$; use `mse_loss` (or BCE with the `Sigmoid` output).
- The code size is your main knob - start around **16-64**.
- **Identity trap**: perfect reconstruction but a useless code $\Rightarrow$ the code is too big. Shrink it, or add noise / an L_1 penalty.
- For images, swap `Linear` for `Conv2d` / `ConvTranspose2d`.

Autoencoders = nonlinear PCA

You already compressed digits once - PCA. This bends the axes.

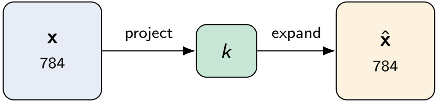
PCA is already an encoder-decoder

Rewind to **PCA** (the dimensionality-reduction chapter). To compress a digit it does two steps:

- **Encode:** project the 784 pixels onto the top k directions $\rightarrow$ k numbers.
- **Decode:** multiply those k numbers back out $\rightarrow$ a reconstruction $\hat{\mathbf{x}}$.

That is *exactly* an autoencoder - only with **linear** encode and decode and no hidden layers.

So PCA is the **simplest possible autoencoder**. The real question: what changes when the encoder and decoder become real (nonlinear) networks?



both arrows are just matrix multiplies

From PCA to a real autoencoder

Keep the encoder and decoder **linear** - the activation is the *identity* $\phi(a) = a$ (just weighted sums, no bend) - and minimize reconstruction error: the autoencoder provably recovers the **same subspace as PCA** - the best flat sheet through the data (directions of maximum variance).

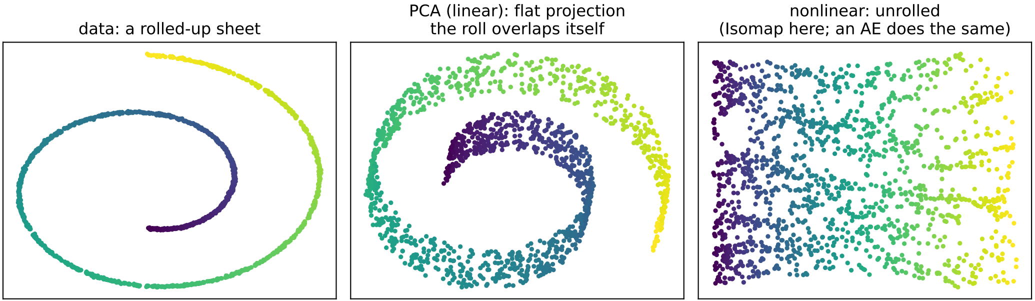
Now switch the activations **on** (ReLU, sigmoid). Encode and decode become **curved**, so the code can follow structure a flat sheet cannot. That is the whole idea:

linear autoencoder = PCA $\longrightarrow$ **nonlinear** autoencoder = PCA that can bend

You do not throw PCA away - you **generalize** it. The next two slides show the difference: first on a curved toy dataset, then on the digits.

See it: PCA flattens, the autoencoder bends

A **swiss roll** - a 2-D sheet rolled up in 3-D. PCA can only take a flat photo of it; a nonlinear method unrolls it back to the sheet:



In PCA's flat photo (middle) far-apart points on the roll **land on top of each other**; the nonlinear map (right; Isomap here, an autoencoder does the same) **unrolls** it so the colour runs smoothly. That is what "bend" buys you.

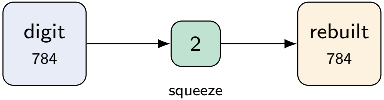
Back to digits: the 2-number experiment

Same task, head to head. Force each test digit through a **2-number** bottleneck, then rebuild it:

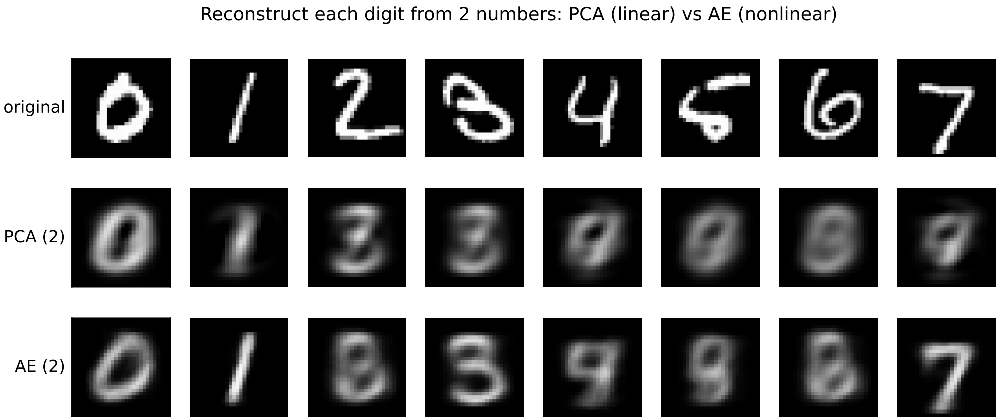
- **PCA**: keep only the top **2 components**, reconstruct (linear).
- **Autoencoder**: a $784 \to 2 \to 784$ net with nonlinear activations.

Both get the *same* 2-number budget; the only difference is linear vs curved.

2 numbers is a brutal squeeze for a 784-pixel digit - neither will be perfect. The question is *which structure survives*.



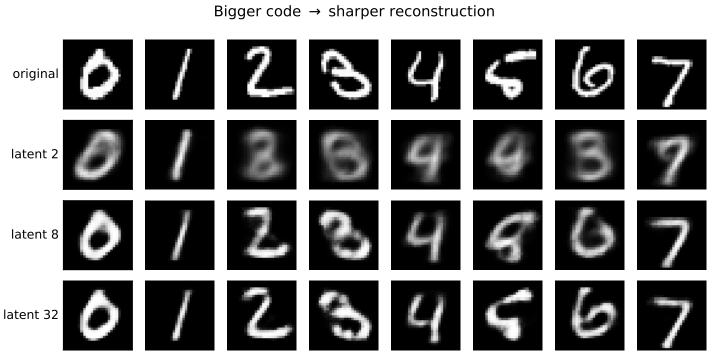
The result: nonlinear keeps more



PCA's 2-component reconstruction (middle row) collapses several digits to a ghostly *average* - 4, 5, 6, 7 all blur into a similar blob. The autoencoder's 2-number code (bottom row) keeps more real digit shape. Same budget - the curve wins.

How big should the code be?

The code size - the **latent dimension** - is the key knob (we used 2 only so we could *draw* the space):

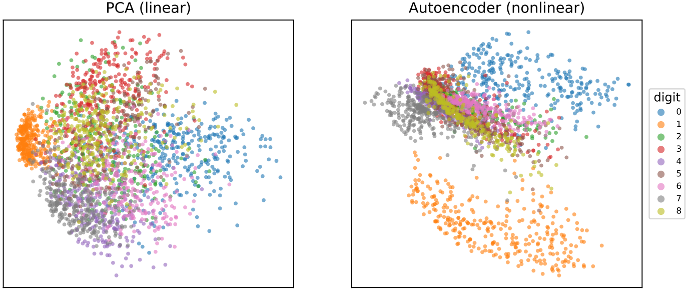


Too small (2): the net **guesses** (a 5 comes back a 9). Bigger (8, 32): sharper, classes separate - but you lose the 2-D picture. Real autoencoders use **tens to hundreds**.

The same digits as a 2-D map

Both methods also give a **2-D map** of the data (each point is one digit).

Honest caveat: the AE's axes are **not** orthogonal, **not** variance-ordered, and **not** unique (another training run gives a different map). PCA hands you all of those for free. The AE trades them for flexibility.



The latent captures more than pixels

That 2-D map is not just pretty - the code captures **structure you can use**.

A striking example from medicine: a patient's **sex** is nearly impossible to read off a raw **brain MRI** - even trained radiologists cannot do it by eye. Yet train an autoencoder on the scans, look at the latent, and **men and women separate**.

The autoencoder surfaced a signal that *was* in the data but hidden from the eye - here enough to fill a missing field in a patient record. That is **representation learning**: a compact code that makes hidden structure explicit. *(Illustrative - a known result on brain imaging, not run here.)*

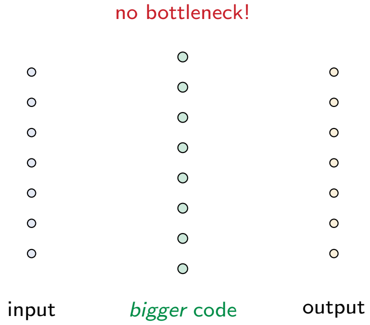
Regularized autoencoders

When you cannot rely on a tight bottleneck, constrain the code another way.

Why go overcomplete?

Compression is not always the goal. Sometimes you want to **re-represent** the input as a *richer* set of features - a code **bigger** than the input, called **overcomplete**. Two reasons it helps:

- **A vocabulary, not a summary.** A language has thousands of words, but any one sentence uses a handful. An overcomplete code is a big **dictionary** of feature-detectors; each image switches on just the **few** it needs (like the brain: many neurons, few firing at once).
- **Spread out to separate.** Structure tangled in the raw pixels can become easy to read once the input is lifted into a bigger, richer space.



…but a big code can cheat

An overcomplete code has room to spare - so the network can take the lazy way out and just **copy the input straight through** (the identity map again), learning nothing useful.

With a big code we can no longer lean on a tight bottleneck to force learning. We need a *different* constraint to keep the network honest. Three classic fixes, coming up: **denoising** (corrupt the input), **sparse** (penalize how much the code fires), and **contractive** (penalize the code's sensitivity to the input).

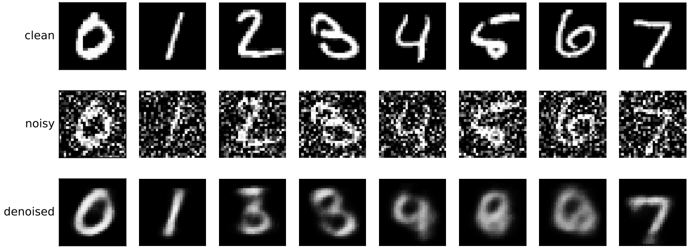
Denoising autoencoders

Break the input on purpose. Corrupt **x** (add noise, or zero out pixels), then train the network to output the **clean** original.

It cannot copy noise, so it must learn what a digit really is.

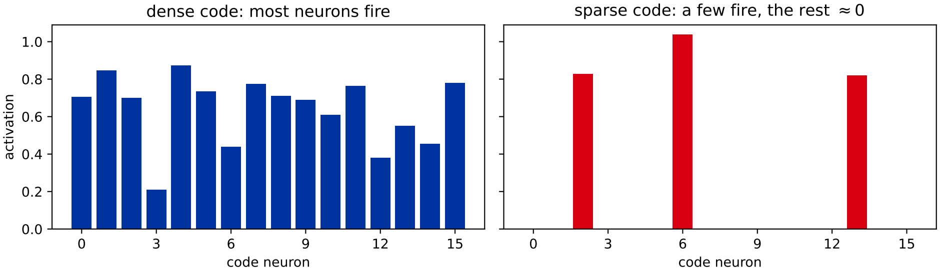
> *Callback:* ch5 data augmentation - robustness by perturbation. Here the perturbation is the whole training signal.

Predict: zero out half the pixels and demand the clean digit back - does that really help? *Yes.*



Sparse autoencoders

Constraint #2: don't shrink the code, **penalize how much it fires**. Add an L_1 penalty on the code activations (or a target average activation), so for any one input only a **few** of the many code neurons switch on.



Each input now lights up a **small, specialised** set of detectors - more interpretable, and it *cannot* copy the input (that would need every neuron). *Callback:* the L_1 penalty is Lasso (ch1), now on **activations** instead of weights.

Contractive autoencoders

Constraint #3: make the code **insensitive to tiny input wiggles**. Penalize how fast the encoder f reacts as the input moves - the squared Frobenius norm of its Jacobian:

$$L\big(\mathbf{x}, g(f(\mathbf{x}))\big) \;+\; \lambda \left\| \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} \right\|_F^2 .$$

Now two nearby inputs map to **almost the same code**: the encoder contracts small perturbations away and keeps only the few directions that actually matter - those along the data manifold.

Denoising vs contractive: the DAE earns a stable code by **corrupting inputs** (sampled noise); the CAE asks for it **directly**, through a deterministic Jacobian penalty. Same goal, two routes. *Callback:* it is just another term added to the loss - exactly like weight decay in ch5.

Anomaly detection

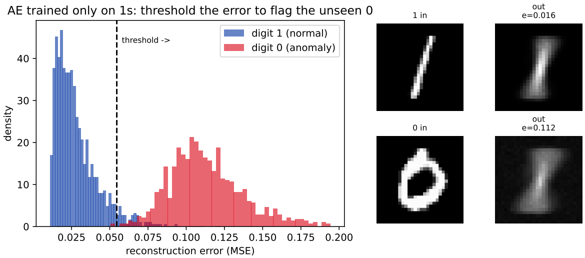
If it cannot rebuild you, you do not belong.

Reconstruction error is an anomaly score

Train an autoencoder on **normal data only**. It learns to rebuild normal inputs with **low** error. Feed it something it has never seen - it rebuilds it **badly**, with **high** error.

So the reconstruction error *is* an anomaly score: pick a **threshold**, flag anything above it.

The industrial workhorse use of autoencoders: fraud, manufacturing defects, network intrusion. *Callback:* thresholding a score - ch3 metrics.



Trained only on 1s; a 0 gets forced toward a 1-shape ⇒ high error.

A caveat you can see

Reconstruction-error anomaly detection only works when the anomaly is **genuinely different** from the training data.

Train on digits 0-8 and try to flag a **9**: it fails - a 9 looks like a 4, a 7, a 1, all of which the network saw, so it reconstructs 9s just fine. The clean split on the previous slide needed a truly distinct anomaly (train on 1s, flag 0s). Know what "normal" and "anomalous" mean for *your* data before you trust the error.

Sparse autoencoders

The same trick, pointed at another network's mind.

Can you read a single neuron?

Take a small **recurrent neural network** (**RNN**, ch7) trained to spell: it reads a stream of words drawn from a **40-word lexicon** and predicts the next character. To finish a word it must carry *which word am I inside* in its **32-dimensional** hidden state.

We generated the text, so we know the true answer at every step. Ask two questions.

Predict first: is "am I inside word #17" written on some neuron?

Can you read a single neuron?

Take a small **recurrent neural network** (**RNN**, ch7) trained to spell: it reads a stream of words drawn from a **40-word lexicon** and predicts the next character. To finish a word it must carry *which word am I inside* in its **32-dimensional** hidden state.

We generated the text, so we know the true answer at every step. Ask two questions.

Predict first: is "am I inside word #17" written on some neuron?

Read it off the best single neuron	F1 = **0.57**
Read it off all 32 together (a trained linear probe)	F1 = **0.99**

The information **is** in the hidden state - a probe finds it almost perfectly. It is just not written on any **one** neuron. The network's own coordinates are the wrong place to look.

Why one neuron is never enough

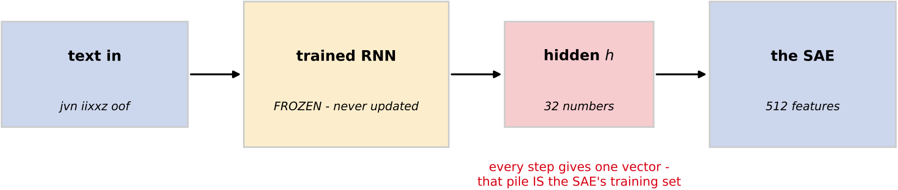
Two reasons, and only the second is exotic.

- **Nothing asks the axes to mean anything.** Gradient descent is free to spread a concept across a *direction* like $0.3\,h_4 - 0.7\,h_{11} + \ldots$. Nothing in the loss rewards lining that direction up with a coordinate axis.
- **Superposition.** Our net must track **40** word identities in **32** dimensions - more concepts than it has neurons. That is only possible because the concepts are **sparse**: at any step you are inside exactly *one* word, so the directions can overlap and rarely collide.

Callback (math, curse of dimensionality): you can pack $e^{c\,p}$ **almost** orthogonal directions into $\mathbb{R}^p$, but only p truly orthogonal ones. High dimensions stop being a curse and start being storage. Real models lean on this hard: a concept lives on a direction, not on a neuron.

The reframe: the activations *are* the data

Every autoencoder so far ate **data** - digit images. A **sparse autoencoder** (**SAE**) eats **another model's hidden vectors**.



Freeze the trained model. Push data through it. Collect the hidden vector **h** at every step. **That pile of vectors is the training set.** Same encoder, decoder and L_1 you already know - only the input distribution changed.

Wide, not narrow

Every other autoencoder in this deck squeezed: $\dim(\mathbf{z}) < \dim(\mathbf{x})$. This one **expands**, typically **4x to 256x** wider than the layer it reads.

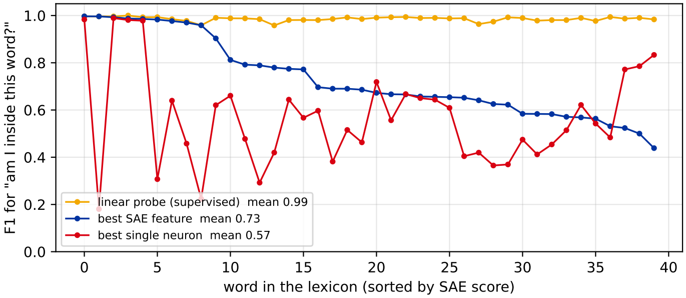
$$\mathbf{z} = \mathrm{ReLU}(\mathbf{W}_{\text{enc}}(\mathbf{h} - \mathbf{b}) + \mathbf{b}_{\text{enc}}), \qquad \hat{\mathbf{h}} = \mathbf{b} + \sum_{i\,:\,z_i>0} z_i\, \mathbf{d}_i$$

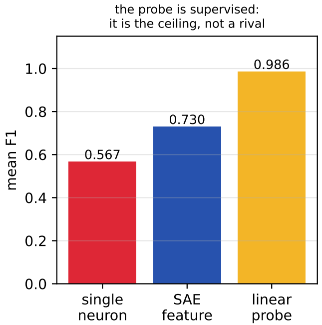
The decoder columns $\mathbf{d}_i$ are **unit vectors**: a **dictionary** of feature directions. Each hidden state is rebuilt from the **few** entries switched on.

$$\mathcal{L} = \underbrace{\|\mathbf{h} - \hat{\mathbf{h}}\|^2}_{\text{stay faithful}} + \lambda \underbrace{\|\mathbf{z}\|_1}_{\text{stay sparse}}$$

So what stops it copying the input? **Not** the width - there is room to spare. Only the **sparsity**. The number that matters is L_0: how many features are on at once.

Does it work? Score it against the truth

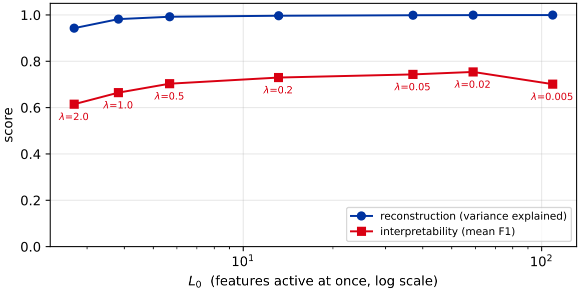




A **512**-feature dictionary on the RNN's 32 numbers, $\lambda = 0.2$, about **13** features active at a time. We generated the corpus, so "is this feature the word *vuxjep*?" has a real answer: this is **scored**, not eyeballed.

The probe is **supervised** - we hand it the labels. The SAE is given **no labels at all** and must find the concept unprompted. The probe is the **ceiling**, not a competitor.

The knob: how sparse is too sparse?



Turn λ up and L_0 falls: fewer features explain each vector.

Reconstruction degrades gently. **Interpretability** does not - it peaks and then falls, because once too few features remain, one feature must cover several concepts again.

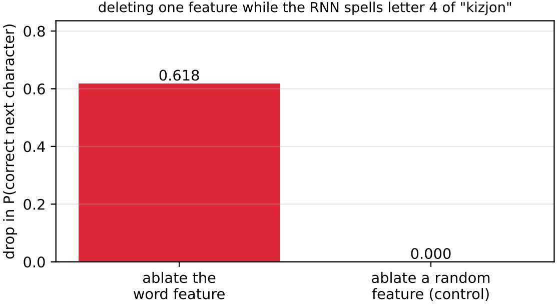
There is no "just maximize sparsity". The useful setting is a **middle** one, and nothing in the loss tells you where it is - the loss cannot see interpretability.

Naming a feature is a guess. Ablating it is a test.

Look at a feature's **top-activating inputs**, see they are all the same word, and you have a **hypothesis**.

To make it a **claim**, intervene: set that one feature to zero, rebuild the hidden state, and let the RNN keep spelling.

Golden Gate Claude (Anthropic, May 2024): they clamped one feature to about **10x** its usual peak and released it publicly for 24 hours. It steered every conversation to the bridge. Same move, bigger model.



Feature #83 fires on the 4th letter of *kizjon*, and nowhere else (F1 $= 1.00$). Delete it and P(correct next character) falls $0.94 \to 0.32$; delete a random feature and it does not move at all. The flat control is what makes this evidence.

Does it scale? And what it still gets wrong

Work	Read from	Features	Year
Towards Monosemanticity	1-layer transformer, 512 neurons	up to ∼13,000	2023
Scaling Monosemanticity	Claude 3 Sonnet	34 million	2024
Scaling and evaluating SAEs	GPT-4	16 million	2024

Not a solved problem. Dead features never fire and waste the dictionary. The L_1 penalty causes **shrinkage**: it drags down the very activations it is measuring. Widen the dictionary and broad features **split** into narrow ones, so the "true" feature count is not well defined. An ICLR 2025 paper is titled, flatly, *Sparse Autoencoders Do Not Find Canonical Units of Analysis*.

Since 2025 the frontier has moved on to **transcoders** and **attribution graphs**, which trace how features *cause* one another rather than only listing which exist.

Recap: autoencoders

- An autoencoder is an **encoder + decoder** trained to rebuild its input - **self-supervised**, no labels.
- The **bottleneck** forces it to keep only structure; a linear one recovers **PCA**, a nonlinear one bends around curved data.
- Cannot rely on a tight bottleneck? Regularize: **denoising** (corrupt and recover), **sparse** (penalize activations), or **contractive** (penalize code sensitivity).
- **Anomaly detection**: high reconstruction error flags what the network never learned.

But it cannot generate

One thing the autoencoder **cannot** do: invent a new digit. To generate, you would draw a random code $\mathbf{z}$ and decode it. But the AE never told its codes to follow any known distribution.

Draw $\mathbf{z} \sim \mathcal{N}(0, I)$ (red) and the codes (blue) are somewhere else entirely: the draws miss almost the whole manifold and decode to blurry blobs. You can **reconstruct**, but you cannot **sample**.

Next (L23): give the latent space no holes, then sample it - **variational autoencoders**.